MPI Parallelization of LBM

1D and 2D splitting

Mouret Basile, Hacini Malik, Boyer Timothé

Grenoble INP - Ensimag, M1 AM

2026-04-29

1. 1D splitting

1.1 Implementations

- **exercise 1** : classical approach using blocking communications `MPI_Send` and `MPI_recv`.
- **exercise 2** : improved using even/odd communication to prevent dependency chain
- **exercise 3** : used non blocking communications `MPI_Isend` and `MPI_Irecv`.

As columns are continuous in memory, we didn't have to use buffer memory to pack the messages.

2. 2D splitting

2.1 Grid initialization

In order to have a clear and usable code, we implemented some helper functions.

- Computing the dimensions of the split : using `MPI_Dims_create` doesn't necessarily yield a split that perfectly divide the length and width of our simulation grid.
- wrote `lbm_choose_2d_split`, that computes the configuration that perfectly divides our grid with cells that are as square as possible.
- we then used `MPI_Cart_create` and `MPI_Cart_coords` to organise our processes and get their coordinates relative to the split.

2.2 Finding neighbooring processes

In order to facillitate finding the rank of neighbooring processes, we used `MPI_Cart_rank` and wrapped it in a small helper function that returns `MPI_PROC_NULL` if the coordinates are out of bound (non peridodic BCs). This is redundant as `MPI_Cart_rank` already does this, but this way it is clear for the users, and implementing other BCs becomes easy.

2.3 Communications

- **exercise 4** : we used the available buffers in `comm` to wrap the non continuous data (up/down)
- **exercise 5 and 6** : created a MPI datatype that fetches the right memory for rows.
- Diagonal communication is done automatically by simply taking the whole line including the ghost cells during top/down communication.
- For non blocking communication (exercise 6) we have to synchronize processes between left/right and up/down using `MPI_Waitall`.

3. Benchmarks

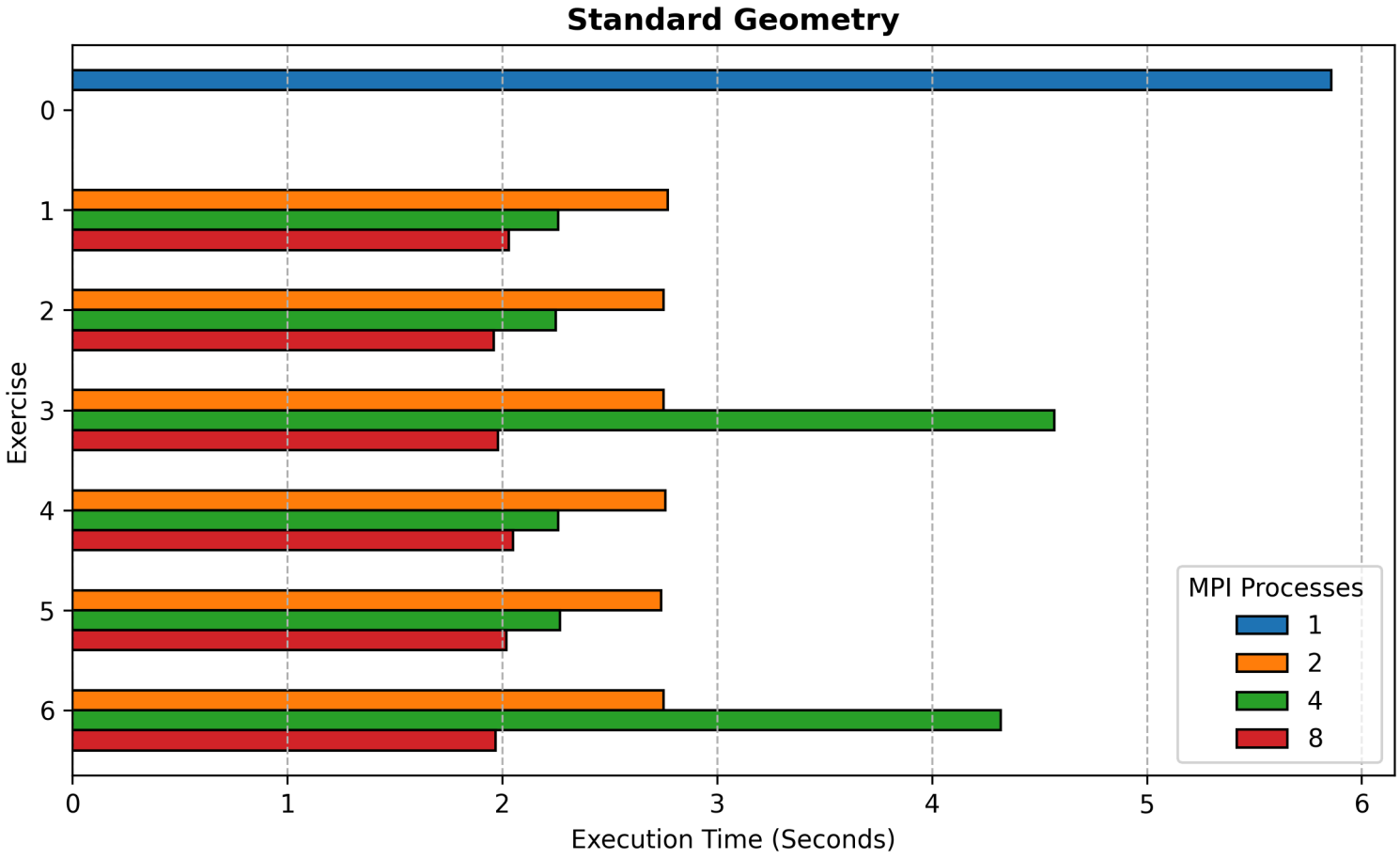
3.1 Configuration

The benchmarks were conducted on our personal machine running Fedora 43:

- 8x Zen 4 cores @4.0 GHz
- 8MB of L2 cache
- 16MB of L3 cache
- 32 Gb of Memory

For each tested case we run each exercise with 2, 4, and 8 processes and the serialized version. We only ran the simulation once as they take a lot of time to run. More accurate benchmarks should be conducted with multiple runs for each case, but this requires a dedicated computing platform.

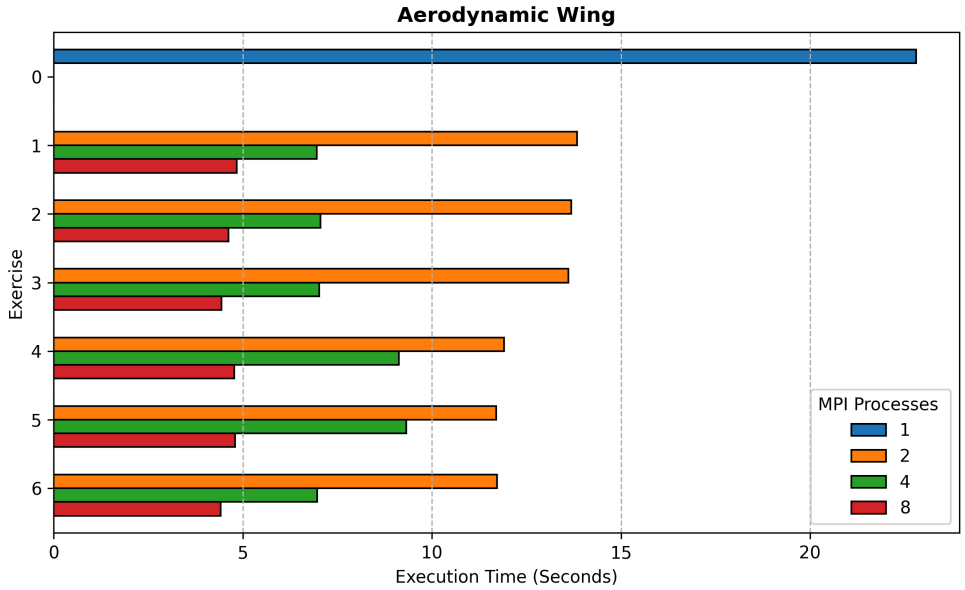
3.2 Default Case

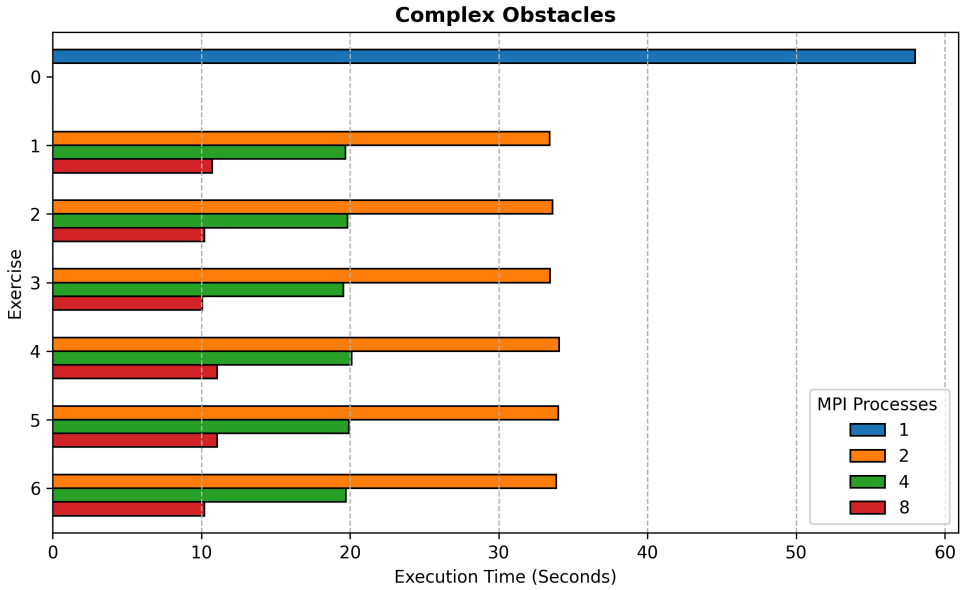


3.2 Default Case

- 2 processes is more then 2x faster then serialized version!
- comparable performances between implementations. In fact the 2d case is the same as 1d, because our splitting puts all the processes on the x axis
- Scaling isn't good as 8 processes is not even 2x faster then 2.
- some discripancy happens for 4 processes in the non blocking versions, we don't know why, but this could be another programm taking resources.

3.3 Other Examples

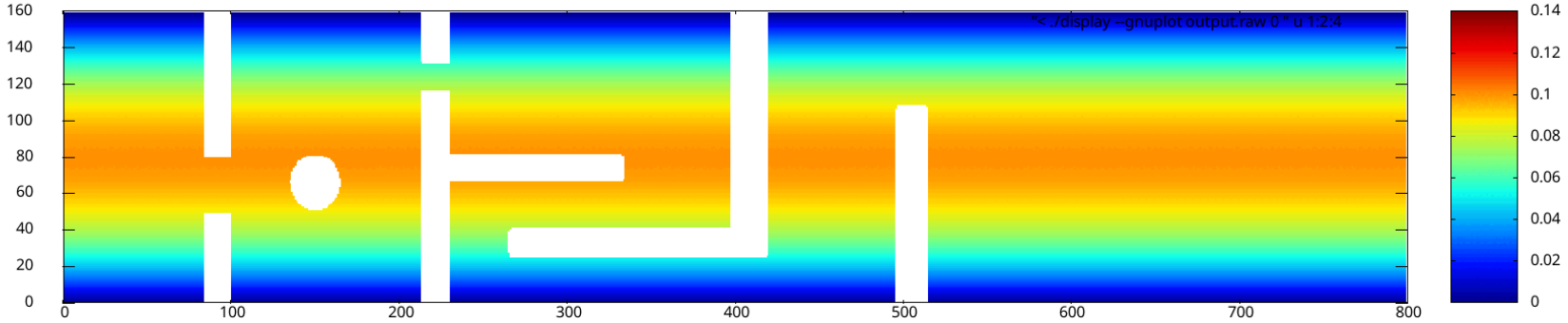


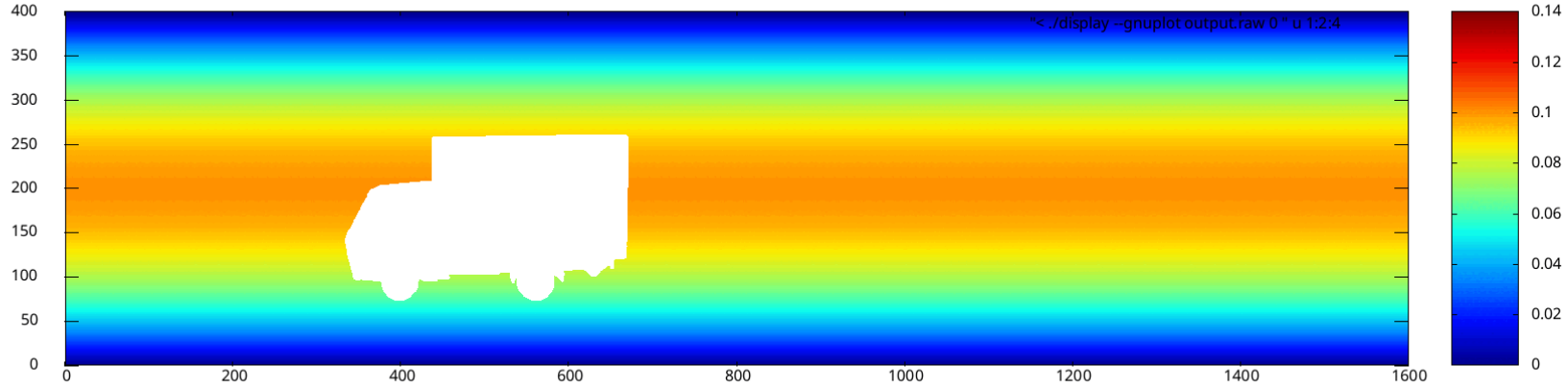


3.4 Other Examples

- Bigger examples than, the default case.
- in the wing example, we see better performance with the 2d splitting.
- scaling is better as the programm becomes more compute bound and the communication overhead becomes negligable
- 8 processes with non blocking 2D splitting method gets us 6x speed increase over the serialized version in both the complex and wing cases.
- the truck example is too big for our benchmarks, we only ran it once for visualizing

3.5 Visuals


"< ./display --gnuplot output.raw 0 " u 1:2:4
160
140
120
100
80
60
40
20
0
0
100
200
300
400
500
600
700
800
0.14
0.12
0.1
0.08
0.06
0.04
0.02
0


"< ./display --gnuplot output.raw 0 " u 1:2:4
400
350
300
250
200
150
100
50
0
0
200
400
600
800
1000
1200
1400
1600
0.14
0.12
0.1
0.08
0.06
0.04
0.02
0